Homework Assignment 1

Solutions

Problem 1 [7pt]

1. Create vectors containing {2.5, 4.5, 6.5, 8.5, 10.5, 12.5, 14.5} elements in three different ways, using the following functions: `c()`, `seq()` with the `by` argument, and `seq()` with the `length.out` argument. Name them `vector1`, `vector2`, and `vector3`, respectively.

```
vector1 <- c(2.5, 4.5, 6.5, 8.5, 10.5, 12.5, 14.5)
vector1
```

```
[1]  2.5  4.5  6.5  8.5 10.5 12.5 14.5
```

```
vector2 <- seq(2.5, 14.5, by = 2)
vector2
```

```
[1]  2.5  4.5  6.5  8.5 10.5 12.5 14.5
```

```
vector3 <- seq(2.5, 14.5, length.out = 7)
vector3
```

```
[1]  2.5  4.5  6.5  8.5 10.5 12.5 14.5
```

2. Append four additional elements to `vector2`: 10, 20, 30, and 40. Ensure that 10 becomes the second element in the updated `vector2`, 20 is the first element considering that 10 has been already added, 30 occupies the 7th position given the presence of 10 and 20, and 40 is positioned as the last element after including all the other numbers.

```
vector2 <- append(vector2, 10, after = 1)
vector2 <- append(vector2, 20, after = 0)
vector2 <- append(vector2, 30, after = 6)
vector2 <- append(vector2, 40)
vector2
```

```
 [1] 20.0  2.5 10.0  4.5  6.5  8.5 30.0 10.5 12.5 14.5 40.0
```

3. Form a new vector encompassing {2, 3, 6, 10, 15, 18, 22, 25, 27, 30, 31, 35, 42} elements and label it `vector4`. Extract elements greater than 15 and not divisible by 2 through a logical statement, generating a new vector named `vector5`. Verify the presence of the number 27 within `vector5` using a logical operator.

```
vector4 <- c(2, 3, 6, 10, 15, 18, 22, 25, 27, 30, 31, 35, 42)
vector4
```

```
 [1]  2  3  6 10 15 18 22 25 27 30 31 35 42
```

```r
vector5 <- vector4[vector4 > 15 & vector4 %% 2 != 0]
vector5
```

```
[1] 25 27 31 35
```

```r
27 %in% vector5
```

```
[1] TRUE
```

Problem 2 [9pt]

1. Create a list `listX` with the following elements:

- A character string "Data Science"
- A numeric value 100
- A vector {-3, 0, 3, 6}
- A logical value `TRUE`
- A sublist containing vectors {"A", "B", "C"} and {2.5, 5.5, 7.5}

```r
listX <- list(
  "Data Science",
  100,
  c(-3, 0, 3, 6),
  TRUE,
  list(c("A", "B", "C"), c(2.5, 5.5, 7.5))
)

listX
```

```
[[1]]
[1] "Data Science"

[[2]]
[1] 100

[[3]]
[1] -3  0  3  6

[[4]]
[1] TRUE

[[5]]
[[5]][[1]]
[1] "A" "B" "C"

[[5]][[2]]
[1] 2.5 5.5 7.5
```

2. Extract the third element of `listX` and assign it to `vector6`.

```r
vector6 <- listX[[3]]
vector6
```

```
[1] -3  0  3  6
```

3. Convert `listX` into a single vector using `unlist()`. Explain why the resulting structure appears as it does.

```
unlist(listX)
```

```
 [1] "Data Science" "100"          "-3"           "0"            "3"
 [6] "6"            "TRUE"         "A"            "B"            "C"
[11] "2.5"          "5.5"          "7.5"
```

By applying `unlist()` to `listX`, we obtain a character vector. From the R documentation for `unlist()`:

> Given a list structure `x`, `unlist()` simplifies it to produce a vector which contains all the atomic components which occur in `x`. Where possible the list elements are coerced to a common mode during the unlisting, and so the result often ends up as a character vector. Vectors will be coerced to the highest type of the components in the hierarchy:
>
> `NULL` < `raw` < `logical` < `integer` < `double` < `complex` < `character` < `list` < `expression`

Under the above hierarchy, the highest data type of an atomic element inside `listX` is "character", so all other elements are coerced to be of class "character" as well. The result thus becomes a character vector.

4. Modify `listX` by replacing the second element with a new value: 200.

```
listX[[2]] <- 200
listX
```

```
[[1]]
[1] "Data Science"

[[2]]
[1] 200

[[3]]
[1] -3  0  3  6

[[4]]
[1] TRUE

[[5]]
[[5]][[1]]
[1] "A" "B" "C"

[[5]][[2]]
[1] 2.5 5.5 7.5
```

5. Create a new list `listY` by merging `listX`with another list containing "Statistics", 50, and `FALSE`. Compare the structures of `listX` and `listY`.

```
listY <- c(listX, list("Statistics", 50, FALSE))
listY
```

```
[[1]]
[1] "Data Science"
```

```
[[2]]
[1] 200

[[3]]
[1] -3  0  3  6

[[4]]
[1] TRUE

[[5]]
[[5]][[1]]
[1] "A" "B" "C"

[[5]][[2]]
[1] 2.5 5.5 7.5


[[6]]
[1] "Statistics"

[[7]]
[1] 50

[[8]]
[1] FALSE
```

The structure of `listY` differs from that of `listX` in that it is longer by 3 elements.

Problem 3 [9pt]

1. Create a data frame `dfA` with the following variables:

- `Team` - {"Alpha", "Beta", "Gamma", "Delta", "Epsilon", "Beta", "Alpha", "Gamma", "Delta", "Epsilon"}
- `Role` - {"Analyst", "Manager", "Developer", "Analyst", "Manager", "Developer", "Analyst", "Manager", "Developer", "Analyst"}
- `Score` - {82, 90, 75, 88, 79, 95, 80, 85, 76, 89}
- `Certified` - {"Yes", "No", "Yes", "No", "Yes", "Yes", "No", "Yes", "No", "Yes"}
- `Experience` - {3, 5, 7, 2, 6, 8, 4, 3, 5, 9}

```
Team <- c("Alpha", "Beta", "Gamma", "Delta", "Epsilon",
          "Beta", "Alpha", "Gamma", "Delta", "Epsilon")

Role <- c("Analyst", "Manager", "Developer", "Analyst", "Manager",
          "Developer", "Analyst", "Manager", "Developer", "Analyst")

Score <- c(82, 90, 75, 88, 79, 95, 80, 85, 76, 89)

Certified <- c("Yes", "No", "Yes", "No", "Yes", "Yes", "No", "Yes", "No", "Yes")

Experience <- c(3, 5, 7, 2, 6, 8, 4, 3, 5, 9)
```

```
dfA <- data.frame(Team, Role, Score, Certified, Experience)
dfA
```

```
      Team      Role Score Certified Experience
1    Alpha   Analyst    82       Yes          3
2     Beta   Manager    90        No          5
3    Gamma Developer    75       Yes          7
4    Delta   Analyst    88        No          2
5  Epsilon   Manager    79       Yes          6
6     Beta Developer    95       Yes          8
7    Alpha   Analyst    80        No          4
8    Gamma   Manager    85       Yes          3
9    Delta Developer    76        No          5
10 Epsilon   Analyst    89       Yes          9
```

2. Compute summary statistics for the `Score` variable.

```
summary(dfA$Score)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  75.00   79.25   83.50   83.90   88.75   95.00
```

3. Convert `Certified` into a factor if it is not already. Display the frequency table of its values.

```
dfA$Certified <- factor(dfA$Certified)
dfA$Certified
```

```
 [1] Yes No  Yes No  Yes Yes No  Yes No  Yes
Levels: No Yes
```

NOTE: The function `as.factor()` also works here:

```
dfA$Certified <- as.factor(dfA$Certified)
dfA$Certified
```

```
 [1] Yes No  Yes No  Yes Yes No  Yes No  Yes
Levels: No Yes
```

```
table(dfA$Certified)
```

```
 No Yes
  4   6
```

4. Add a new column `High_Performer`, which is `TRUE` if `Score` is greater than 85 and `Experience` is more than 4, otherwise `FALSE`.

```
dfA$High_Performer <- dfA$Score > 85 & dfA$Experience > 4
dfA$High_Performer
```

```
 [1] FALSE  TRUE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE  TRUE
```

NOTE: Your first instinct might be to use `ifelse()` for this question, like so:

```r
dfA$High_Performer <- ifelse(dfA$Score > 85 & dfA$Experience > 4, TRUE, FALSE)
```

This is not incorrect. However, using `ifelse()` is redundant if you want to create a logical vector (i.e. a vector containing `TRUE` and `FALSE` values) – so instead of writing `x <- ifelse(condition, TRUE, FALSE)` you can just write `x <- condition`.

5. Extract all employees who belong to the "Gamma" team and are certified.

Solution #1 (using dataframe extraction)

```r
dfA[dfA$Team == "Gamma" & dfA$Certified == "Yes", ]
```

```
   Team      Role Score Certified Experience High_Performer
3 Gamma Developer    75       Yes          7          FALSE
8 Gamma   Manager    85       Yes          3          FALSE
```

Solution #2 (using the `subset()` function)

```r
subset(dfA, Team == "Gamma" & Certified == "Yes")
```

```
   Team      Role Score Certified Experience High_Performer
3 Gamma Developer    75       Yes          7          FALSE
8 Gamma   Manager    85       Yes          3          FALSE
```

Problem 4 [9pt]

1. Generate a matrix named `matrix1` with 5 columns, filled row-wise with the elements: 10, 3, 6, 23, -5, -4, 13, 17, 5, 6, -7, -10, 13, 39, 20, 2, 1, 9, 11, -22, 23, -15, -3, 6, 12.

```r
elements <- c(10, 3, 6, 23, -5, -4, 13, 17, 5, 6, -7, -10,
              13, 39, 20, 2, 1, 9, 11, -22, 23, -15, -3, 6, 12)

matrix1 <- matrix(elements, ncol = 5, byrow = TRUE)
matrix1
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]   10    3    6   23   -5
[2,]   -4   13   17    5    6
[3,]   -7  -10   13   39   20
[4,]    2    1    9   11  -22
[5,]   23  -15   -3    6   12
```

2. Employ a `for` loop to update the elements of `matrix1` according to the following criteria: replace negative values with 0; for elements greater than or equal to 0 but less than 5, replace them with 5; for elements less than 15 but greater than 10, replace them with 10; for elements that do not meet the specified conditions, substitute them with 100.

```r
for (i in 1:nrow(matrix1)) {

  for (j in 1:ncol(matrix1)) {

    if (matrix1[i, j] < 0) {
      matrix1[i, j] <- 0
```

```
    } else if (matrix1[i, j] >= 0 & matrix1[i, j] < 5) {
      matrix1[i, j] <- 5
    } else if (matrix1[i, j] > 10 & matrix1[i, j] < 15) {
      matrix1[i, j] <- 10
    } else {
      matrix1[i, j] <- 100
    }
  }
}

matrix1
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]  100    5  100  100    0
[2,]    0   10  100  100  100
[3,]    0    0   10  100  100
[4,]    5    5  100   10    0
[5,]  100    0    0  100   10
```

3. Initiate an empty vector `vector7`. Utilize a `while` loop to fill `vector7` with odd numbers greater than 0 and less than 30, excluding those odd numbers divisible by 3.

```
vector7 <- vector(mode = "numeric")

i <- 1

while (i < 30) {
  if (i %% 2 == 1 & i %% 3 != 0) {
    vector7 <- c(vector7, i)
  }
  i <- i + 1
}

vector7
```

```
 [1]  1  5  7 11 13 17 19 23 25 29
```

Problem 5 [9pt]

1. Write a function `categorize_numbers()` that takes a numeric vector as input and classifies each element into one of three categories:

- `"Small"` if the number is less than 10
- `"Medium"` if the number is between 10 and 20 (inclusive)
- `"Large"` if the number is greater than 20

The function should return a character vector with the corresponding category labels. Test your function with the vector: {4, 10, 15, 22, 8, 18, 27, 5, 12}.

Solution #1 (Using a `for` loop)

```
categorize_numbers <- function(vec) {
  labels <- vector(mode = "character", length = length(vec))
  for (i in 1:length(vec)) {
```

```
    if (vec[i] < 10) {
      labels[i] <- "Small"
    } else if (vec[i] >= 10 & vec[i] <= 20) {
      labels[i] <- "Medium"
    } else {
      labels[i] <- "Large"
    }
  }
  labels
}
```

```
test_vec <- c(4, 10, 15, 22, 8, 18, 27, 5, 12)

categorize_numbers(test_vec)
```

```
[1] "Small"  "Medium" "Medium" "Large"  "Small"  "Medium" "Large"  "Small"
[9] "Medium"
```

Solution #2 (without using a `for` loop)

```
categorize_numbers <- function(vec) {
  labels <- ifelse(vec < 10, "Small", ifelse(vec >= 10 & vec <= 20, "Medium", "Large"))
  labels
}
```

```
test_vec <- c(4, 10, 15, 22, 8, 18, 27, 5, 12)

categorize_numbers(test_vec)
```

```
[1] "Small"  "Medium" "Medium" "Large"  "Small"  "Medium" "Large"  "Small"
[9] "Medium"
```

NOTE: This solution is a lot more compact because it uses the `ifelse()` function instead of `if` statements. An `if` statement only works on one element at a time, but `ifelse()` can deal with a whole vector (or a whole matrix) at once. This can helps us avoid `for` loops, making our code a lot shorter and simpler.

Notice the use of two `ifelse()` functions, one nested inside the other.

The first `ifelse()` corresponds to the `if` part that checks whether an element is less than 10:

> **If the element is less than 10** return "Small" as the corresponding category label.
>
> **If the element is NOT less than 10** we enter the second `ifelse()`. This corresponds to the `else if` part that checks whether an element is between 10 and 20 (inclusive).
>
> > **If the element is between 10 and 20 (inclusive)** return "Medium" as the category label.
> >
> > **If the element is NOT between 10 and 20 (inclusive), meaning that it is greater than 20,** return "Large" 10 as the category label. This final piece corresponds to what the `else` part was handling.

2. Create a function `matrix_summary()` that accepts a numeric matrix and returns a list containing the following elements:

- `Row Sums` - a vector with the sum of each row

- `Column Means` - a vector with the mean of each column
- `Max Value` - the maximum value in the entire matrix
- `Min Value` - the minimum value in the entire matrix

Hint: `sum()`, `mean()`, `max()`, `min()` functions will come in handy.

Test your function on the matrix:

$$\begin{bmatrix} 5 & 12 & 18 \\ 7 & 14 & 22 \\ 9 & 10 & 25 \\ 6 & 17 & 30 \end{bmatrix}$$

Solution #1

```
matrix_summary <- function(mat) {
  # row sums
  row_sums <- vector(mode = "numeric", length = nrow(mat))
  for (i in 1:nrow(mat)) {
    row_sums[i] <- sum(mat[i, ])
  }
  # column means
  col_means <- vector(mode = "numeric", length = ncol(mat))
  for (i in 1:ncol(mat)) {
    col_means[i] <- mean(mat[ , i])
  }
  # maximum value
  max_val <- max(mat)
  # minimum value
  min_val <- min(mat)
  # package everything up in a list
  summary <- list(
    "Row Sums" = row_sums,
    "Column Means" = col_means,
    "Max Value" = max_val,
    "Min Value" = min_val
  )
  summary
}
```

```
test_mat <- matrix(c(5, 12, 18, 7, 14, 22, 9, 10, 25, 6, 17, 30), ncol = 3, byrow = TRUE)

matrix_summary(test_mat)
```

```
$`Row Sums`
[1] 35 43 44 53

$`Column Means`
[1]  6.75 13.25 23.75

$`Max Value`
[1] 30

$`Min Value`
[1] 5
```

Solution #2

R has many in-built functions that simplify common operations, so you might have stumbled upon the `rowSums` and `colMeans` functions (the corresponding functions `rowMeans` and `colSums` also exist).

```
matrix_summary <- function(mat) {
  summary <- list(
    "Row Sums" = rowSums(mat),
    "Column Means" = colMeans(mat),
    "Max Value" = max(mat),
    "Min Value" = min(mat)
  )
  summary
}
```

```
test_mat <- matrix(c(5, 12, 18, 7, 14, 22, 9, 10, 25, 6, 17, 30), ncol = 3, byrow = TRUE)

matrix_summary(test_mat)
```

```
$`Row Sums`
[1] 35 43 44 53

$`Column Means`
[1]  6.75 13.25 23.75

$`Max Value`
[1] 30

$`Min Value`
[1] 5
```